

PRACTICAL WEEK-3

Session-3:

Task -1

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

CODE:

```
> OSSP_Programs > Practical > Week3 > C task1.c
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4
5  int main() {
6
7      pid_t pid;
8
9      printf("Before fork()\n");
10     printf("PID : %d\n", getpid());
11     printf("PPID : %d\n\n", getppid());
12
13     pid = fork();
14
15     if (pid < 0) {
16         printf("Fork failed.\n");
17     }
18
19     else if (pid == 0) {
20         // Child process
21         printf("Child Process\n");
22         printf("PID : %d\n", getpid());
23         printf("PPID : %d\n", getppid());
24     }
25
26     else {
27         // Parent process
28         printf("Parent Process\n");
29         printf("PID : %d\n", getpid());
30         printf("Child PID : %d\n", pid);
31     }
32
33     return 0;
34 }
```

OUTPUT :

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ cd /mnt/d/OSSP_2520080069/Practical/Week3
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week3$ ls
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week3$ nano task1.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week3$ gcc task1.c -o task1
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week3$ ./task1
Before fork()
PID : 761
PPID : 317

Parent Process
PID : 761
Child PID : 762
Child Process
PID : 762
PPID : 761
```

OBSERVATION:

The program successfully created a child process using `fork()`. The parent process has PID 761, while the child process has PID 762. The child's PPID is 761, confirming that the child was created by the parent. Both processes continued execution after `fork()`, demonstrating the creation and identification of parent and child processes.

Task -2

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

CODE:

```
D: > OSSP_Programs > Practical > Week3 > C task2.c
1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      printf("Process started.\n");
6      printf("PID: %d\n", getpid());
7
8      printf("Process is running...\n");
9
10     printf("Process entering waiting state for 10 seconds...\n");
11     sleep(30);
12
13     printf("Process resumed running.\n");
14
15     printf("Process will terminate now.\n");
16
17     return 0;
18 }
```

OUTPUT :

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week3$ nano task2.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week3$ gcc task2.c -o task2
ananya@Ananya:/mnt/d/OSSP_2520080069/Practical/Week3$ ./task2
Process started.
PID: 807
Process is running...
Process entering waiting state for 10 seconds...
Process resumed running.
Process will terminate now.
```

OBSERVATION:

The C program was executed as a Linux process and its state was monitored using ps, top, and /proc. The process entered a sleeping/waiting state during sleep(10) and resumed execution afterward. Once the program completed, the process terminated and was no longer listed by ps.